Offensive Container Security: Techniques, Misconfigurations, and Attack Paths

Explore real-world misconfigurations, container escape techniques, Kubernetes privilege escalation, CI/CD attack vectors, and persistent threats.

Updated September 26, 2025 • 15 min read

Rushikesh

Contents

Introduction:

In this post, I present a series of offensive techniques and real-world misconfigurations encountered while exploring containerized environments, particularly Docker and Kubernetes. From container escapes via mounted sockets to privilege escalation within clusters, this research highlights the impact of seemingly minor oversights. Security practitioners and red teamers can use these insights to better understand attack paths and secure their container workloads more effectively.

Docker Environment Threats and Misconfigurations:

- **Privileged Mode Abuse:**

  Running a container with `--privileged` grants it nearly all host capabilities (e.g. `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`) and disables Linux security controls. An attacker in a privileged container effectively has root on the host.

  In practice, privileged containers are often used by CI systems (e.g. Docker-in-Docker, Jenkins), but they pose a critical risk: an exploit in a privileged container can modify kernel modules or the host filesystem to gain full host control.

  Any sensitive workload (production or CI) using `--privileged` greatly widens the attack surface. Security best practices strongly advise against privileged containers except when absolutely necessary.

- **Exposed Docker Socket (var/run/docker.sock):**

  The Docker socket on the host listens on `/var/run/docker.sock` and effectively gives root-equivalent API access. If this socket is mounted into a container (or exposed over the network) the container can issue Docker commands to control the host. For example, an attacker could run `docker run -v /:/host --privileged` and then `chroot /host sh` to gain a shell on the host filesystem.

Studies note that **"if anyone obtains access to Docker socket, they have permissions equivalent to root on host"**. Indeed, Docker itself warns that installing applications as a non-root user or restricting socket access is critical, since an exposed socket allows arbitrary host commands.

Red teams commonly exploit this: by mounting the host's root (`/`) into a malicious container and chrooting, attackers can modify `/etc/passwd` or run `chroot` to own the host. Scanners like **Trivy** and **kubescape** flag exposed sockets as high-risk misconfigurations.

- **Host Filesystem Mounts:**

  Even without using the socket, Docker volumes can mount host directories. For instance, `docker run -v /:/mnt ... chroot /mnt` is a classic escalation: the container's attacker mounts the host root and performs a `chroot` inside, effectively jumping from the container to host root context.

  Any misconfigured volume mapping (e.g. bind-mounting `/etc` or `/home` from the host) can expose sensitive host files to the container. Best practices dictate minimizing or avoiding hostPath mounts, or marking them read-only, to prevent attackers from planting backdoors on the host.

- **Linux Capabilities and Privileges:**

  Docker containers can be given extra Linux capabilities (e.g. `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`, `CAP_DAC_OVERRIDE`). Granting unnecessary capabilities is dangerous; for example, `CAP_SYS_ADMIN` is often described as a "God-like" capability allowing many privileged operations. TrendMicro notes that containers with these capabilities "have all root capabilities of the host".

  A creative escalation involves enabling the setuid bit on a binary inside a container that is also mounted on the host: if the container shares the host user namespace, a setuid binary (e.g. new `root` user entry) can elevate privileges on the host.

  Attackers and tools like DEEPCE exploit combinations of capabilities and setuid files to break out. To mitigate, one should drop unnecessary capabilities and use

Docker's `--security-opt` (e.g. AppArmor, seccomp) profiles.

- **User Namespace and Root Mapping:**

  By default, containers run processes as root mapped to host root, meaning a compromised container root can be host root. Enabling Docker **user namespaces** remaps container root to an unprivileged host UID, reducing risk. However, not all deployments use user namespaces by default. Without userns, any successful container escape (e.g. via a kernel exploit) results in host root compromise. Administrators should strongly consider userns or at least avoid processes running as root inside containers.

- **Docker API Exposure and CVEs:**

  If Docker's remote API is enabled (e.g. listening on TCP without TLS), unauthorized access is possible. Recent CVEs highlight new risks: for instance, **CVE-2024-41110** is a high-severity Docker Engine authorization bypass found in July 2024.

  A specially crafted API request (with `Content-Length: 0`) could evade AuthZ plugins and let an attacker run unauthorized commands. This regression (fixed in Docker 18.09 but lost in later releases) was finally patched in mid-2024.

  Other known Docker CVEs include **CVE-2019-5736** (runc container escape) and several BuildKit/Dockerfile parser issues (CVE-2024-23651/52/53). Attackers can exploit these by **building or pulling malicious images**. Docker's own advisory warns that **"running a container from a suspect image"** could give unauthorized host filesystem access or even complete container escape. Administrators must patch Docker Engine, runc, and BuildKit promptly to eliminate such vulnerabilities.

Kubernetes-Specific Risks:

- **Exposed Kubelet Endpoints:**

The **kubelet** is the per-node agent that manages pods and containers. By default, it listens on port 10250 (HTTPS) and exposes diagnostics and exec APIs. If kubelet's API is reachable (especially without authentication) an attacker can effectively control the node. Research shows that **hundreds of thousands** of kubelet endpoints are internet-accessible, and dozens allow anonymous access.

Aqua Security found ~287,000 connected kubelets, and at least 100 of them were open to unauthenticated requests. In lab attacks, compromised kubelets allowed adversaries to launch new containers on the node, steal secrets, and deploy cryptocurrency miners.

For example, TeamTNT's campaigns scan for Kubelet (port 10250) and use it to spawn mining containers.

Cloud-native defenders should ensure kubelet is not exposed: enable HTTPS/TLS, require client certs or token auth, and restrict network access.

- **Service Account Token Abuse:**

  Every Kubernetes pod normally mounts a service account (SA) JWT token (by default the `default` SA in the namespace) at `/var/run/secrets/kubernetes.io/serviceaccount`. This token can call the Kubernetes API with the pod's permissions.

  Attackers who compromise a pod (e.g. via an escape) can steal its SA token and use it for lateral movement. Insecure or overly-permissive SA bindings are common pitfalls. A token with cluster-wide rights is especially dangerous. KubeOps warns that **"if an attacker compromises a pod, they can use the default ServiceAccount token to interact with the Kubernetes API, escalating privileges and potentially compromising the entire cluster"**.

  Red teams often exploit this by exfiltrating mounted tokens and then using `kubectl` or API calls to create new pods, escalate roles, or harvest secrets. Best practice is to disable auto-mount of SA tokens when not needed (especially for non-controller pods) and to apply strict least-privilege RBAC.

- **Insecure RBAC Configurations:**

Kubernetes RBAC (Role-Based Access Control) defines which identities can perform which actions. However, many clusters have misconfigurations or over-permissive roles. For instance, a RoleBinding meant for a single namespace may mistakenly use a ClusterRole, granting broader access than intended. Attackers exploit such misconfigurations to escalate privileges. As SentinelOne notes, **"attackers typically exploit permissive RBAC roles, targeting those that are overly permissive or too broadly applied"**.

A common pattern is to break into any pod in the cluster and then use its service account (or impersonate it) to bind a cluster-admin role, gaining full control. Defenders should audit all Roles/ClusterRoles and their bindings. Techniques such as segmenting namespaces and regularly reviewing `kubectl get clusterrolebinding` help catch these issues.

- **Malicious DaemonSets and Workload Abuses:**

  A DaemonSet ensures its pods run on every node. If an attacker gains the ability to create workloads, they often use a DaemonSet to quickly propagate a payload cluster-wide. For example, Aqua's research observed a campaign deploying a malicious XMRig crypto-miner via a DaemonSet (image `lchaia/xmrig`), which amassed over 1.4 million pulls on Docker Hub.

  Another advanced attack crafted a DaemonSet that mounted each host's filesystem and installed a cron job, ensuring persistence on every node.

  Since DaemonSets automatically cover all nodes, they are a prime target for persistence. Other similar vectors include Jobs and CronJobs: an attacker can schedule recurring malicious tasks. Red Hat's security guide warns that Kubernetes controllers (like DaemonSets and CronJobs) **can be abused to ensure a container is always running somewhere in the cluster**.

  To mitigate, restrict RBAC so only trusted admins can create DaemonSets/CronJobs, and monitor for anomalous workload creations.

- **GitRepo Volume Vulnerabilities (CVE-2024-10220, CVE-2025-1767):**

Kubernetes' deprecated `gitRepo` volume type has seen multiple serious flaws. In CVE-2024-10220, an attacker with permission to create pods using a `gitRepo` volume could execute **arbitrary commands on the host**. This works because `git clone` can run Git hooks on the node itself; a malicious repo with a crafted `post-checkout` hook was able to run as root on the host.

Upwind Research explains that gitRepo volumes "execute git clone commands without sanitizing the contents... hooks execute in the host environment", effectively turning a pod creation into a host compromise. A follow-on flaw, CVE-2025-1767, lets an attacker create a pod with a `gitRepo` volume that references any path on the node, allowing theft of arbitrary files (though not code execution). Since `gitRepo` is enabled by default (despite deprecation), any cluster allowing pod creation by untrusted users should disable this feature. Both vulnerabilities underscore how Kubernetes features assumed to be "in-cluster" can unexpectedly touch the node.

Container Escape and Privilege Escalation Techniques:

Recent container runtime bugs demonstrate how attackers can break isolation and gain host access. For instance, CVE-2019-5736 allowed a malicious runc payload to execute as root on the host.

More recently, **CVE-2024-21626** (a runc bug) and several BuildKit issues were disclosed; Docker's advisory warns that running malicious builds or images *"could lead to full container escape"*. In practice, attackers may exploit these via specially crafted Dockerfiles or by pulling tainted images.

Tools like DEEPCE automate container enumeration and include many breakout primitives. Unit42 demonstrates that techniques such as abusing `cgroup release_agent` (setting a special file that executes on cgroup unload) or `ptrace` injection can pop a shell on the host.

For example, DEEPCE's "release_agent" exploit uses a setuid binary trick: the container writes a setuid file into a host-shared directory and triggers it via cgroups,

granting host root. In summary, without patched runtimes and careful configuration, **container escapes** remain a primary privilege escalation vector.

Beyond kernel/runc bugs, basic Linux escalation applies if an attacker lands on the host. Standard post-exploitation enumeration scripts like **LinPEAS** can hunt for world-writable mounts, insecure SUID files, or stolen credentials to escalate. However, many container attacks simply pivot by taking over the orchestration tools: once host root is obtained, the attacker typically steals all service account tokens on the host (e.g. in `/etc/kubernetes`), enabling cluster-level compromise. In essence, container breakout often leads immediately to full cluster takeover if any pod's token has privileges.

Persistence and Post-Exploitation in Clusters:

Once inside a host or cluster, adversaries set up persistence. Common techniques include:

- **Malicious DaemonSets and ReplicaSets:**

  Attackers create DaemonSets (or Deployments with a pod affinity) to ensure their payload runs on every node or always stays running. In one documented campaign, the threat actor repeatedly launched crypto-miner containers via a DaemonSet and even used a host-mounted DaemonSet to plant crontab entries on each node.

  This guarantees that even if one pod is killed, others restart or reappear. Since DaemonSets run at system scope, any malicious container in a DaemonSet effectively has node-wide reach.

- **HostPath Backdoors:**

  Exploiting hostPath volumes or mounts can let attackers persist on the host filesystem. For example, mounting a `/etc` or `/usr/bin` directory (via hostPath) into a container could allow the attacker to drop scripts or binaries that survive pod restarts. Red Hat notes that hostPath mounts "allow an attacker to persist on

the container host". Defenders should restrict hostPath volumes via Pod Security Policies or K8s admission controls.

- **Kubernetes CronJobs:**

  A **CronJob** can schedule tasks (containers) on a timer. An attacker with privileges can create a CronJob that periodically launches a malicious pod (e.g. for data exfiltration or command-and-control beacons). This is analogous to cron on Linux. Red Hat identifies this as an ATT&CK persistence technique (3.3) and recommends locking down permissions to create CronJobs.

- **Hidden or Disguised Containers:**

  Malicious pods may be given innocuous names or labels to hide among normal workloads. For example, TeamTNT used common base images (like Alpine variants) with slight name changes, accumulating tens of thousands of pulls.

  Attackers also sometimes annotate pods to appear as monitoring tools. Regular audits (e.g. with `kubectl get pods --all-namespaces`) and workload-label baselining can help detect these.

In sum, the persistence stage often leverages normal K8s primitives. Guardian mechanisms (monitoring runtime activity, endpoint detection like Aqua/Falco, etc.) should alert on unusual DaemonSet or CronJob creations.

CI/CD Pipeline Abuse and Image Supply Chain Risks:

Container platforms do not stop at runtime. The build and deployment pipeline also offers attack opportunities:

- **Image Supply Chain Attacks:**

  Malicious or tampered container images are a key vector. **Typosquatting** on image names is common; attackers publish images with names nearly identical to popular ones (e.g. `nginx` vs `nginxx`) containing crypto miners or backdoors.

Sysdig's threat research found hundreds of malicious Docker Hub images masquerading as legitimate services and hiding cryptocurrency miners.

Similarly, **dependency confusion** can occur if internal image names overlap with public ones. Base images (e.g. `alpine`, `ubuntu`) have occasionally been mirrored with malicious code – any automated build pulling an unpinned latest image risks infection.

- **Embedded Secrets in Images:**

  Developers sometimes mistakenly bake secrets (SSH keys, API tokens) into container layers. In a Sysdig analysis of 250k images, secrets (credentials, keys) were the second-most common malicious payload after cryptominers.

  Any secret found in an image can give attackers immediate access (for example, an SSH key in an image lets the attacker hop into the environment). Scanners like Trivy now include secret scanning to flag this risk. Embedding secrets violates the principle of immutable build artifacts, so CI/CD systems must actively scan all images and commits for leaked credentials.

- **Poisoning Base Images and CI/CD Jobs:**

  Attackers may compromise trusted registries or staging servers. A famous example (beyond containers) was the SolarWinds supply-chain incident; similarly, a malicious change in a widely-used Dockerfile or CI configuration can compromise many downstream builds.

  For instance, if an image builder (like Docker Build Cloud) is given a malicious frontend (CVE-2024-23651/52/53), build artifacts can be tainted.

  Defense in CI/CD includes pinning images to digests, using image signing (Docker Content Trust or Sigstore/Cosign), and verifying build steps. Docker recommends *"Only use trusted images (such as Docker Official Images)"* and avoid building from untrusted Dockerfiles.

- **CI/CD Server Compromise:**

An attacker might breach a CI server (Jenkins, GitLab, etc.) which often has Docker and kubectl access. For example, a pipeline job running as root with Docker-in-Docker privileges could be hijacked to launch malicious containers.

Modern attacks have shown malicious commits or merge requests injecting new pipeline stages that harvest credentials or open reverse shells. Rigorous access controls, immutable logs, and requiring code review on pipeline definitions help mitigate this.

In practice, **image security tooling** is essential. Open-source scanners like Trivy and Clair can detect known vulnerabilities and misconfig (including CVEs in container software, embedded secrets, and outdated libraries). Organizations should integrate these scanners into their CI/CD to catch supply-chain issues before deployment.

Threat Actor Campaigns and Case Studies:

Several threat groups and publicized campaigns illustrate these risks:

- **TeamTNT Kubelet Exploits:**

  Since 2021, the TeamTNT group has scoured the internet for open Kubelet and Docker APIs, then deployed cryptominers and API-stealing malware. Their `scanrand()` function randomly finds IP ranges with open port 10250 (Kubelet) and then uses `pwnkube()` to exploit them.

  The payloads often drop Monero miners inside containers. Aqua observed TeamTNT using trojanized container images (e.g. `lacy/nginxpm` on Docker Hub) that look benign but contain rootkits. This underscores the danger of both exposed endpoints and supply-chain (malicious images).

- **Lchaia/Xmrig Crypto Campaign:**

  In 2023, researchers spotted a massive mining campaign where attackers pushed 1.4M pulls of `lchaia/xmrig:latest` on Docker Hub.

Using a single YAML, they deployed this miner as a DaemonSet across thousands of clusters. The campaign even leveraged Tor for launch. Similarly, the "ssww" campaign created a DaemonSet that mounted host filesystems and wrote cron jobs on every node, ensuring the miner restarted. These cases show how rapidly an image pull can escalate to global infection when DaemonSets are abused.

- **CVE-2024-10220 Exploit (GitRepo):**

  The critical gitRepo flaw has been demonstrated in live breaches. An attacker with minimal permissions can spin up a pod using a `gitRepo` volume pointing to a malicious repo. Upon `git clone`, the repo's hooks execute on the **host** (not the container), giving host-level code execution. Upwind's lab confirmed full host takeover can result, highlighting that even **deprecated** features (like gitRepo) remain exploitable if not disabled.

- **RBAC Misuse (RBAC Buster):**

  Recent research (Aqua Nautilus's "RBAC Buster" campaign) found attackers quietly creating hidden resources. One example was a campaign using stolen credentials to bind a rogue `ClusterRole` to a low-visibility service account, then deleting audit logs. While details are still emerging, it illustrates how over-permissive RBAC can let attackers embed backdoors.

These incidents make clear that adversaries combine these techniques: initial access may come via a misconfigured kubelet or CI secret leak, and once inside they deploy persistent miners or credential harvesters via DaemonSets and hostPath mounts. Monitoring community publications (Unit42, Aqua, etc.) and sharing threat intelligence on forums helps organizations stay abreast of new tactics.

Security Tools and Resources:

To defend against these container/cluster attacks, security teams employ specialized tools and training:

- **Container Escape Scanners:**

DEEPCE is a shell-based tool that enumerates Docker misconfigurations and even demonstrates escape exploits (e.g. by enabling `release_agent`). It automates many of the manual steps attackers use.

- **Vulnerability and Secret Scanners:**

  Trivy (by Aqua) and other scanners (Clair, Anchore, Snyk) can scan images for CVEs in packages, outdated components, and *embedded secrets* (API keys, certificates). Trivy's secret scanning mode will flag hard-coded credentials in layers, directly addressing issues seen by Sysdig.

- **Kubernetes Security Auditors:**

  Tools like *kube-hunter* and *kube-bench* (CIS Benchmark) check clusters for common misconfigurations (e.g. anonymous Kubelet access, hostPath usage, etc.). *Kubesec* and *polaris* can lint manifests for insecure settings.

- **Runtime Detection:**

  Platforms like Falco or Aqua Nautilus can detect suspicious container behavior (e.g. a container spawning a shell or mounting host paths). Some extended cloud detection rules now specifically look for containers creating host files or calling `/chroot`.

- **Penetration Testing & Labs:**

  Community labs help practitioners understand these flaws: the **Kubernetes Goat** project is a "vulnerable by design" K8s cluster for hands-on exploitation. On Hack The Box, machines like **SteamCloud** (Easy) simulate misconfigured Kubernetes clusters for red-teamers. These labs cover many techniques discussed here (e.g. taking over kubelets, abusing RBAC and DaemonSets). Training through such exercises familiarizes defenders with offensive tactics.

Red teamers also rely on general Linux tools (e.g. *LinPEAS*, *pspy*, *chrono* malware tests) inside containers to find escalation paths, and *kubectl* or *kubeletctl* (a pentesting CLI) to interact with the cluster from a compromised pod. Staying current with threat research blogs and CVE advisories is critical, given the rapid disclosure of container-specific bugs in the last two years.

References:

This report cites current threat intelligence and research up to May 2025. Key sources include official advisories (Docker, Kubernetes), security lab blogs (Palo Alto Unit42, Aqua Nautilus, Snyk, Sysdig), and industry publications.)

https://trendmicro.com

https://flast101.github.io/docker-privesc/

https://aquasec.com

https://redhat.com

https://unit42.paloaltonetworks.com

https://raesene.github.io/blog/2025/03/14/cve-2025-1767-another-gitrepo-issue/

https://www.armosec.io/

#kubernetes #docker #appsec #offensive-security #devsecops

More from this blog

Protocol Zero: Securing Agentic AI and the Model Context Protocol (MCP)

A Security Architecture for Autonomous Agents, From Prompt Injection to Remote Code Execution

Feb 6, 2026 · 23 min read

Subscribe to the newsletter

Get new posts delivered to your inbox.

you@example.com Subscribe

Fast-Flux & Double Fast-Flux: Anatomy, Detection, and Defensive Response

Dive deep into fast-flux and double fast-flux DNS techniques. Learn how botnets evade detection, how to detect them using DNS metrics, and how to defe

Oct 4, 2025 · 25 min read

Data Exfiltration in Modern Environments: A Comprehensive Threat and Defense Handbook

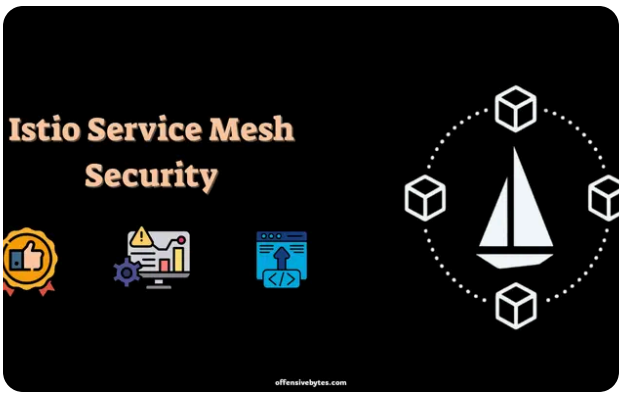
Explore every known data exfiltration technique, from network tunnels to AI-assisted attacks, with actionable detection rules and prevention methods.

Sep 10, 2025 · 41 min read

Istio Service Mesh Security: Best Practices, Misconfigurations, and Real-World Deployment

A deep technical dive into Istio architecture, mTLS, policy enforcement, and securing Kubernetes microservices

Aug 21, 2025 · 29 min read



Attackers Love Your YAML: Static Kubernetes Security Analysis for DevSecOps

Shift left, stay secure: Identifying and fixing the most dangerous Kubernetes misconfigurations in CI/CD.

Jun 30, 2025 · 15 min read

OffensiveBytes

12 posts published

Decoding the Art of Offensive Security

© 2026 OffensiveBytes